SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1404 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO3: *Analyz syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree.* *(BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:100**

Annexure A

EXPERIMENTS

Code of Conduct:

I **CHINTHAPARTHI NAVEEN REDDY / 192525206***(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.
C. Naveen Reddy
Signature of the student:

Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 30**	**/ 20**	**/ 15**	**/ 10**	**/ 100**

Annexure A

EXPERIMENTS

Q1. Write a Lex program to recognize uppercase letters (A–Z) from the given input. The program should identify and count all capital letters present in the string. Display the total number of uppercase letters detected.

Q2Write a Lex program to validate an email address entered by the user. The program should check for the presence of a valid username, '@' symbol, and domain name. Display whether the email address is valid or invalid.

Q3. Write a Lex program to recognize and validate a date of birth in the format DD/MM/YYYY. The program should check whether the entered date follows the specified format. Display a message indicating whether the date of birth is valid or invalid.Q4. You are building a type checker in a compiler.

Q4. Write a Lex program to perform token recognition in a source program. The program should identify keywords, identifiers, operators, separators, and numeric constants. Display the type of each token recognized from the input.

Q5.Write a Lex program to identify vowels and consonants in a given string. The program should count the number of vowels and consonants separately. Display the total counts after processing the input string.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1:

UPPERCASE LETTER RECOGNITION

Aim:

To develop a Lex program that identifies and counts all uppercase letters (A–Z) present in a given input string.

Algorithm:

1. Start the program.
2. Read the input string from the user.
3. Scan each character in the input.
4. Check whether the character is an uppercase letter (A–Z).
5. If it is uppercase, display the character and increment the count.
6. Continue until the end of the input.
7. Display the total number of uppercase letters.
8. Stop.

C-Program:

```
%{
#include <stdio.h>
int uppercase_count = 0;
%}
%%
[A-Z]   {
          printf("Uppercase Letter Detected: %s\n", yytext);
          uppercase_count++;
        }
.|\n    { /* Ignore all other characters */ }
%%
```

```
int yywrap() {
    return 1;
}
int main() {
printf("==========================================\
n");
    printf("  EXP 1: UPPERCASE LETTER RECOGNITION\n");
printf("==========================================\
n");
    printf("Enter string: ");
    yylex();
    printf("\nTotal Uppercase Letters Detected = %d\n",
uppercase_count);
    return 0;
}
```

Sample Input:

NAVEENREDDY

Sample Output:



Result:

The program successfully recognizes all uppercase letters in the input string and displays their total count.

EXPERIMENT 2:

EMAIL ADDRESS VALIDATION

Aim:

To develop a Lex program that validates an email address by checking the username, '@' symbol, domain name, and extension.

Algorithm:

1. Start the program.
2. Read the email address from the user.
3. Check whether the email contains a valid username.
4. Verify the presence of the '@' symbol.
5. Check for a valid domain name and extension.
6. If all conditions are satisfied, display Valid Email.
7. Otherwise, display Invalid Email.
8. Stop.

C-Program:

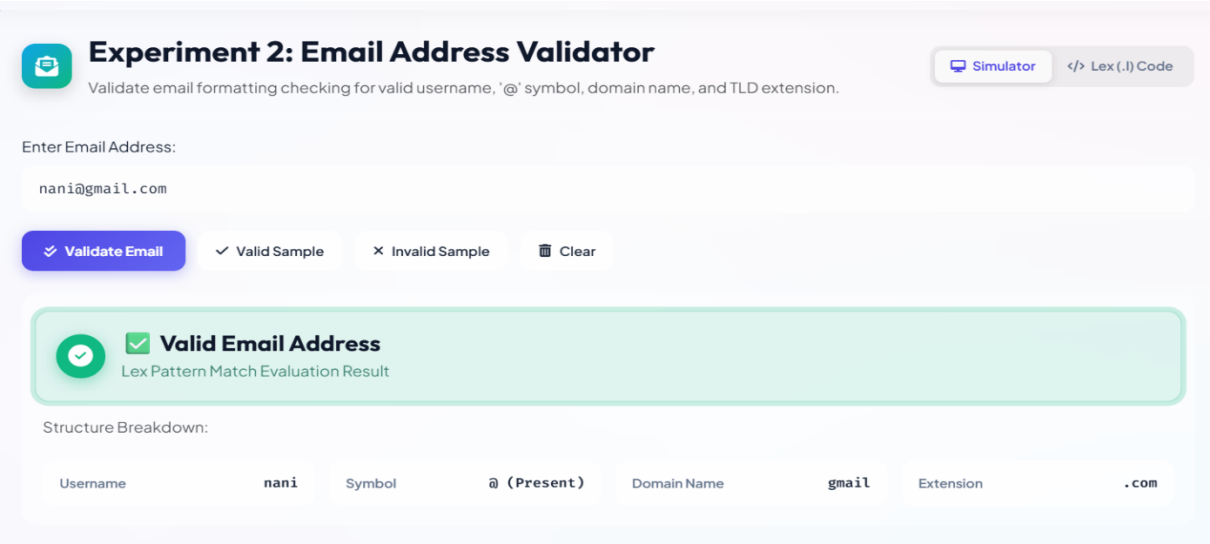
```
%{
#include <stdio.h>
int is_valid = 0;
%}
%%
[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}    { is_valid =
1; }
.|\n                                { /* Catch invalid input */ }
%%
int yywrap() {
    return 1;
}
```

```
int main() {
printf("==========================================\
n");
    printf("       exp 2: email validator\n");
printf("==========================================\
n");
    printf("Enter Email Address: ");
    yylex();
    if(is_valid) {
        printf("\n Valid Email Address\n");
    } else {
        printf("\n Invalid Email Address\n");
    }
    return 0;
}
```

Sample Input:

nani@gmail.com

Sample Output:



Result:

The program correctly validates the entered email address and displays whether it is valid or invalid.

EXPERIMENT 3:

DATE OF BIRTH VALIDATION

Aim:

To develop a Lex program that validates a date of birth in the format DD/MM/YYYY.

Algorithm:

1. Start the program.
2. Read the date from the user.
3. Check whether it follows the DD/MM/YYYY format.
4. Verify that the day, month, and year are in valid ranges.
5. If the format is correct, display Valid Date of Birth.
6. Otherwise, display Invalid Date of Birth.
7. Stop.

C-Program:

```
%{
#include <stdio.h>
#include <stdlib.h>
int is_valid = 0;
%}

%%
([0-2][0-9]|3[01])\/(0[1-9]|1[0-2])\/([12][0-9]{3}) {
    int day = atoi(yytext);
    int month = atoi(yytext + 3);
    int year = atoi(yytext + 6);

    // Leap year check logic:
```

```
   int isLeap = (year % 4 == 0 && year % 100 != 0) || (year % 400
== 0);
   int daysInMonth[] = {0, 31, isLeap ? 29 : 28, 31, 30, 31, 30, 31,
31, 30, 31, 30, 31};

   if (day >= 1 && day <= daysInMonth[month]) {
      is_valid = 1;
      printf("Parsed Date: Day=%02d, Month=%02d, Year=%d
(Leap Year: %s)\n",
             day, month, year, isLeap ? "Yes" : "No");
   }
}
.|\n    { /* Ignore invalid input */ }
%%

int yywrap() {
   return 1;
}
int main() {

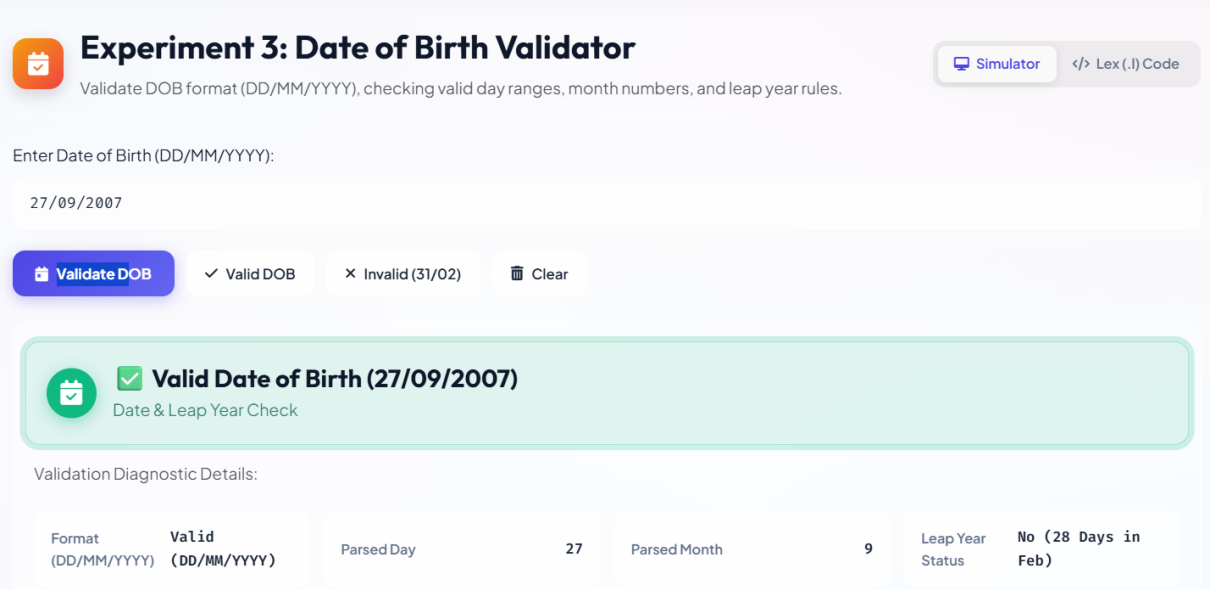
printf("=========================================\
n");
   printf("    exp 3: date of birth validator\n");
printf("=========================================\
n");
   printf("Enter Date of Birth (DD/MM/YYYY): ");
   yylex();

   if(is_valid) {
      printf("\n Valid Date of Birth\n");
   } else {
      printf("\n Invalid Date of Birth\n");
   }
   return 0;
}
```

Sample Input:

27/09/2007

Sample Output:



Result:

The program verifies whether the entered date of birth follows the specified format and displays the validation result.

EXPERIMENT 4:

LEX TOKEN RECOGNIZER

Aim:

To develop a Lex program that recognizes different types of tokens in a source program.

Algorithm:

1. Start the program.
2. Read the source program.
3. Scan the program token by token.
4. Identify keywords.
5. Identify identifiers.
6. Identify operators.
7. Identify separators.
8. Identify numeric constants.
9. Display the type of each recognized token.
10. Stop.

C-Program:

```
%{
#include <stdio.h>
int token_count = 0;
%}
%%
"int"|"float"|"char"|"double"|"void"|"if"|"else"|"while"|"for"|"return
" {
   printf("Token: %-15s | Type: Keyword\n", yytext);
   token_count++;
}
```

```
[a-zA-Z_][a-zA-Z0-9_]* {
    printf("Token: %-15s | Type: Identifier\n", yytext);
    token_count++;
}
[0-9]+(\.[0-9]+)? {
    printf("Token: %-15s | Type: Numeric Constant\n", yytext);
    token_count++;
}
"=="|"!="|"<="|">="|"&&"|"||"|"++"|"--"|"+"|"-
"|"*"|"/"|"%"|"="|">"|"<" {
    printf("Token: %-15s | Type: Operator\n", yytext);
    token_count++;
}
";"|","|"{"|"}"|"("|")"|"["|"]" {
    printf("Token: %-15s | Type: Separator\n", yytext);
    token_count++;
}
"//".* {
    printf("Token: %-15s | Type: Comment\n", yytext);
    token_count++;
}
[ \t\n]+ { /* Skip Whitespaces */ }

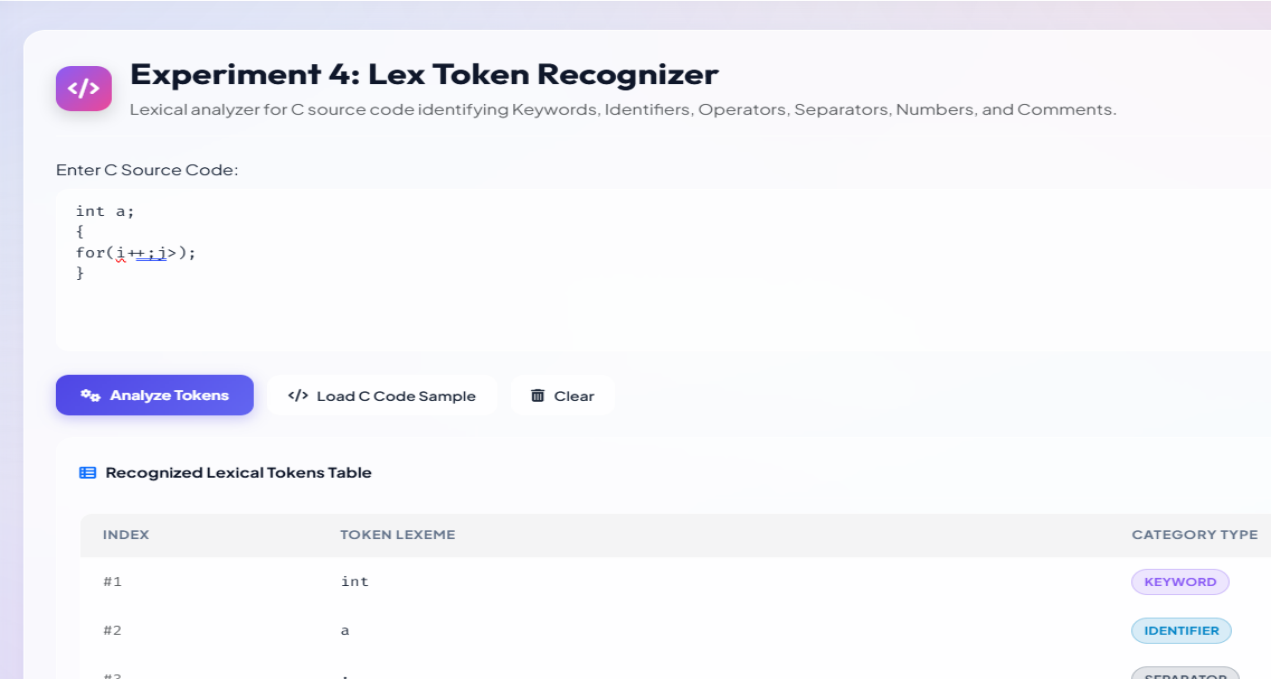
. {
    printf("Token: %-15s | Type: Unknown\n", yytext);
    token_count++;
}
%%
int yywrap() {
    return 1;
}
int main() {
```

```
printf("==========================================\
n");
    printf("      exp 4: lex token recognizer\n");
printf("==========================================\
n");
    printf("Enter C Source Program Code:\n\n");
    yylex();
    printf("\nTotal Tokens Recognized = %d\n", token_count);
    return 0;
}
```

Sample Input:

```
Int a;
{
For(i++:j>);
}
```

Sample Output:



Result:

The program successfully recognizes and classifies tokens such as keywords, identifiers, operators, separators, and numeric constants.

EXPERIMENT 5:
VOWELS AND CONSONANTS COUNTER

Aim:

To develop a Lex program that counts the number of vowels and consonants in a given string.

Algorithm:

1. Start the program.
2. Read the input string.
3. Scan each character.
4. If the character is a vowel, increment the vowel count.
5. If the character is a consonant, increment the consonant count.
6. Ignore spaces, digits, and special characters.
7. Display the total number of vowels and consonants.
8. Stop.

C-Program:

```
%{
#include <stdio.h>
int vowel_count = 0;
int consonant_count = 0;
%}

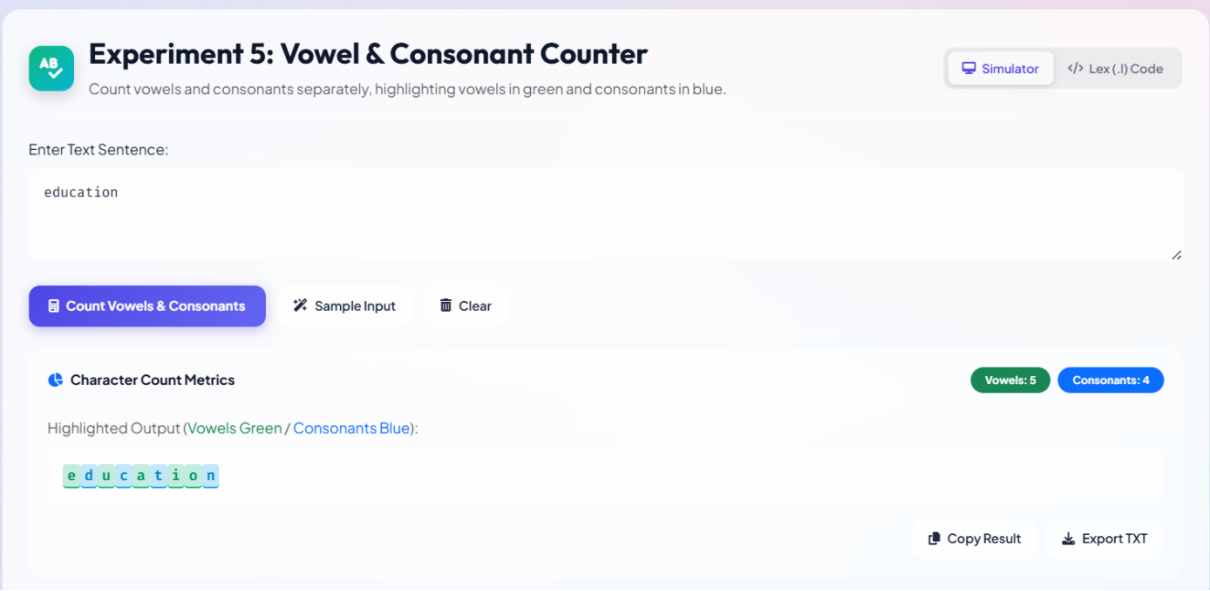
%%
[aeiouAEIOU]                    { vowel_count++; }
[b-df-hj-np-tv-zB-DF-HJ-NP-TV-Z]    { consonant_count++; }
.|\n                        { /* Ignore numbers & special symbols */
}
%%
int yywrap() {
   return 1;
}
```

```
int main() {
printf("==========================================\
n");
    printf("   exp 5: vowel and consonant counter\n");
printf("==========================================\
n");
    printf("Enter string: ");
    yylex();
    printf("\n---------------------------------------\n");
    printf("Total Vowels     = %d\n", vowel_count);
    printf("Total Consonants = %d\n", consonant_count);
    printf("---------------------------------------\n");
    return 0;
}
```

Sample Input:

Education

Sample Output:



Result:

The program correctly counts and displays the total number of vowels and consonants present in the input string.